



T.N.S.V.T

Sistema Copy

Documentación Técnica Integral

Reino del Cristo Íntegro

Trading Neuro-Spiritual Value Theory · 2026

Backend Symfony 8.1 · PHP 8.4 · SQLite · Mercure · Firebase FCM

Signal Copier Python 3.12 · Telethon · MetaTrader5 · FastAPI · Streamlit

2 Apps Android · Capacitor v8 (Web) + v6 (Market Instinct)

Tailscale Funnel · <https://laptop-ebgqig6j.tailf43f87.ts.net/>

Versión 1.0 · Fecha: 06/07/2026

Confidencial · Uso interno TNSVT

Repos: github.com/federicocasal73-gif/tnsvt-symfony · github.com/federicocasal73-gif/TNSVT-SISTEMA-COPY

§1 Resumen Ejecutivo

Visión del Proyecto

T.N.S.V.T (Trading Network for Strategic Visionary Traders) es un ecosistema integrado de trading algorítmico, educación financiera y gestión de comunidad, distribuido en dos repositorios coordinados: el backend Symfony (PHP) que sirve a dos aplicaciones Android nativas, y el sistema Python que opera como signal copier automatizado conectado a MetaTrader 5. Ambos sistemas se comunican vía API REST, con heartbeat continuo de estado y sincronización de trades en tiempo real.

Componentes Principales

- **Backend Symfony 8.1 (PHP 8.4)** — 29 entidades, 38 controladores, 9 servicios, 115+ endpoints REST, 15 migraciones, modo admin y usuario con CodeAuthenticator
- **Signal Copier Python 3.12** — Telethon (Telegram listener), MetaTrader5 (ejecutor), SQLite local, parser multi-formato, risk manager, news filter
- **Telegram Bot** — 11 comandos, integración TNSVT API, heartbeat 30s, 4 servicios externos (markets, news, calendar, trading_economics)
- **TNSVT Bridge (FastAPI)** — server async en port 8502, cola asyncio.Queue, auth HMAC, integración con PHP via X-Admin-Password
- **Streamlit Dashboard** — 4 tabs (Dashboard, Operaciones, Riesgo, Configuración), JetBrains Mono dark theme
- **Web APK (com.tnsvt.app)** — Capacitor v8, v1.6.x, Academia + Journal + Chat + Social + Chart
- **Game APK (T.N.S.V.T Market Instinct)** — Capacitor v6, v1.2.x, 8 modos de juego + Duelos 1v1 + XP/rankings
- **Tailscale Funnel** — Acceso público HTTPS vía <https://laptop-ebgqig6j.tailf43f87.ts.net/>
- **Pagos compartidos** — MercadoPago (ARS) + Binance Pay (USDT), webhook verification con HMAC
- **Push notifications** — FCM v1 API (OAuth2 service account) + Legacy fallback, 14 tipos de notificación
- **Tiempo real** — Mercure Hub (Docker, port 3000) + SSE EventSource para velas 15m streaming

Métricas Globales

Entidades	29
Controllers	38
Servicios	9
Endpoints REST	115+
Líneas JS	6,271
APKs	2

Comandos Console	8
Migraciones BD	15
Comandos Bot	11
Servicios Python	13
Stack modes	8
URL pública	✓

§2 Arquitectura del Sistema Unificado

El sistema sigue una arquitectura cliente-servidor distribuida. El backend Symfony expone una API REST JSON consumida por ambos APKs Android y por el sistema Python. El signal_copier Python se comunica con TNSVT vía HTTP para auto-journal de trades, heartbeat de estado y configuración remota desde el admin panel.

Diagrama Unificado



Flujo End-to-End de una Señal

- 1. Origen** — Canal de Telegram publica señal (BUY/SELL EURUSD con SL/TP)
- 2. Listener** — signal_copier/main.py recibe via Telethon, parser identifica tipo
- 3. Risk check** — NewsFilter consulta calendario económico, RiskManager valida límites
- 4. Ejecución MT5** — MT5Executor envía orden, magic number 20260706 identifica origen
- 5. Auto-Journal TNSVT** — tnsvt_client.log_trade() crea Trade entity con notes='Auto-copied from: <channel>'
- 6. Trade Monitor** — mt5_trade_monitor detecta cierre (history_deals_get), actualiza PnL en TNSVT
- 7. Heartbeat continuo** — Status JSON se actualiza cada 30s, visible en admin Copier dashboard
- 8. Telegram Bot paralelo** — tnsvt_status.py expone /stats y /senales via TNSVT API

§3 Stack Tecnológico Completo

Backend PHP (Symfony)

Componente	Tecnología
Framework	Symfony 8.1.*
PHP	>= 8.4
ORM	Doctrine ORM 3.6 + Migrations 4.0
Database (dev)	SQLite
Database (prod)	PostgreSQL
Security	Custom CodeAuthenticator + PasswordHasher
CORS	NelmioCorsBundle
Mailer	Symfony Mailer (null://null en dev)
Real-time	Mercure Hub Bundle + Docker
HTTP Client	Symfony HttpClient
Serialization	Symfony Serializer + PropertyAccess
Logging	Monolog + MonitorEvent entity
Templating	Twig 3.x + Symfony Asset Mapper

Frontend Web (PWA)

Componente	Tecnología
HTML/CSS	Twig + CSS modular (~3,000 líneas)
JavaScript	Vanilla JS modular (~6,271 líneas)
API Client	Custom fetch wrapper (api.js, 326 líneas)
Charts	Lightweight Charts (TradingView) v4.2.1
PWA	Service Worker v57 con cache-first + offline
Biometrics	@aparajita/capacitor-biometric-auth v10
Push Web	Firebase Web SDK (VAPID)
Cache-bust	v=timestamp query params
Importmap	Symfony Asset Mapper

Sistema Python (Signal Copier)

Componente	Tecnología
Python	3.12
Telegram Listener	Telethon (asyncio)
MT5 Integration	MetaTrader5 (official lib)
Database	SQLite local (trades.db)
Telegram Bot	python-telegram-bot v20.3
Bridge HTTP	FastAPI + uvicorn[standard]
Dashboard	Streamlit + pandas
APIs externas	requests + python-dotenv
TradingEconomics	tradingeconomics SDK
Encoding	qrcode[pil]

Aplicaciones Android

Componente	Web APK	Game APK
Framework	Capacitor v8	Capacitor v6
Package	com.tnsvt.app	com.tnsvt.market.instinct
Versión	v1.6.x (actual 1.6.3)	v1.2.x
Tamaño	268 MB	5.22 MB
JDK	21.0.7+6	21.0.7+6
Build	gradlew assembleDebug	gradlew assembleDebug
Contenido	Frontend web completo	HTML5 canvas game
APK path	public/downloads/tnsvt-app.apk	public/downloads/tnsvt-market-instinct.apk
Firebase	google-services.json	google-services.json

Servicios Externos

Servicio	Uso
MetaTrader 5	Ejecución real de trades en cuenta demo/real
Telegram API	Telethon escucha canales, Bot responde a usuarios
MercadoPago	Pagos en ARS (pesos argentinos) — Checkout Pro
Binance Pay	Pagos en USDT — HMAC-SHA512 signature

Servicio	Uso
Binance API	Datos de mercado / klines (público)
Firebase FCM	Push notifications — service account JWT v1 API
Mercure Hub	Real-time events via Docker — SSE streaming
TradingEconomics	Calendario económico + indicadores macro
Yahoo Finance	Forex/indices/stock prices (resumen mercados)
DolarAPI	Argentina exchange rates (ARS / USD)
JBlanked News	Calendario económico alternativo
NewsAPI	Búsqueda de noticias (IPC, Morgan, etc.)
Tailscale	Funnel HTTPS público + acceso remoto seguro

§4 Backend TNSVT Symphony (PHP/Symfony)

El backend es un monolito Symfony 8.1 modular organizado por dominio funcional. Sirve a múltiples clientes (2 APKs Android, PWA web, sistema Python) con autenticación unificada vía X-Game-Code header o session web.

Estructura del Proyecto

```
tnsvt-symfony/ ■■■ assets/ ■■■ app.js # Frontend principal (6,271 líneas) ■■■ api.js # API client (326 líneas)
■■■ chart.js # Chart TradingView + Mercure SSE ■■■ styles/ ■■■ app.css # ~3,000 líneas ■■■ web-glowup.css
# Animaciones + cards premium ■■■ apk-layout-fix.css ■■■ apk-glowup.css ■■■ config/ ■■■ packages/ # Bundles
config (doctrine, security, etc.) ■■■ routes.yaml ■■■ docker-compose.yml # Mercure hub ■■■ docs/ # PDFs de
documentación ■■■ migrations/ # 15 migraciones ■■■ public/ ■■■ sw.js # Service Worker v57 ■■■ assets/ # JS/CSS
compilados ■■■ icons/ # PWA icons + manifest ■■■ uploads/avatars/ ■■■ downloads/ # APK files ■■■ apk/ #
Versioned APK backups ■■■ src/ ■■■ Command/ # 8 comandos console ■■■ Controller/ # 38 controllers
(Api/Admin/Main) ■■■ Entity/ # 29 Doctrine entities ■■■ Repository/ # Doctrine repositories ■■■ Security/ #
CodeAuthenticator, UserProvider ■■■ Service/ # 9 servicios de negocio ■■■ templates/ ■■■ base.html.twig # SPA
template único ■■■ emails/ # Twig email templates ■■■ var/ ■■■ data_dev.db # SQLite (dev) ■■■ log/ # Logs +
emails debug ■■■ composer.json ■■■ .env
```


29 Entidades Doctrine (por dominio)

Las entidades están organizadas por dominio funcional. Aquí el detalle de cada una:

4.1 Núcleo de Usuarios y Auth

Entidad	Tabla	Campos clave	Relaciones
User	users	id, code(unique), name, email, active, roles(json), password, walletBalance, lastLogin	OneToMany: trades, tournaments, feed, conversations, devices, notifications, journal
TraderProfile	trader_profiles	strategy, style, favoritePairs, riskPerTrade, experience, extraNotes	OneToOne → User
Device	devices	fcmToken(512), platform(32), deviceModel(128), registeredAt, lastSeenAt	ManyToOne → User (unique fcmToken)

4.2 Trading y Posiciones

Entidad	Tabla	Campos clave	Relaciones
Trade	trades	id, date, asset, direction(long/short), entry, sl, tp, result(win/loss), pnl, ratio, notes, photos(json)	ManyToOne → User
TradingAccount	trading_accounts	id, name, broker, accountNumber, balance, currency, isDeleted	OneToMany → Trade
Tournament	tournaments	name, entryFee, prizePool, prizeDistribution(60/30/10), startDate, endDate, status, maxPlayers(100)	ManyToOne → User(createdBy), OneToMany → TournamentEntry
TournamentEntry	tournament_entries	startingEquity, finalEquity, pnlUsd, pnlPct, finalRank, payoutAmount, status	ManyToOne → Tournament + User (unique)
Duel	duels	code(DUEL-XXXX), entryFee, prizePool, totalRounds(5), currentRound, player1Pnl, player2Pnl, startingPrice, status	ManyToOne → User(p1, p2, winner) + OneToMany → DuelRound
DuelRound	duel_rounds	roundNumber, player1Move, player2Move, openPrice, closePrice, highPrice, lowPrice, player1Pnl, player2Pnl	ManyToOne → Duel

4.3 Wallet y Transacciones

Entidad	Campos	Tipos (TYPE_*)
WalletTransaction	type, amount, currency(USD), refPaymentId, status, notes	DEPOSIT, ENTRY_FEE, PAYOUT, REFUND, WITHDRAW, DUEL_ENTRY, DUEL_WIN, DUEL_REFUND

Status: PENDING, CONFIRMED, REJECTED, REFUNDED. **Methods:** MANUAL_MP, MANUAL_BINANCE, AUTO_MP, AUTO_BINANCE, GIFT, OTHER.

4.4 Feed Social y Mensajería

Entidad	Tabla	Función
FeedPost	feed_posts	Post en feed con content, category, likes, comments(json), signal(json), photo
LikedPost	liked_posts	Registro de likes (ManyToMany implícito)
Message	messages	Mensaje de chat con content, photo, isAi, metadata(json), editedAt, attachment(json)
Conversation	conversations	type(group/dm/ai), title, aiUserCode, lastMessageAt
ConversationParticipant	conversation_participants	lastReadAt, joinedAt

4.5 Academia y Progreso

Entidad	Tabla	Función
AcademiaContent	academia_content	Cursos: title, subtitle, emoji, description, videoUrl, orden, locked, lessons(json)
ModuleProgress	module_progress	moduleId, completed (progreso del usuario)
Task	tasks	title, description, orden, active (tareas operativas)
TraderProfile	trader_profiles	strategy, style, favoritePairs, riskPerTrade

4.6 Journal Social y Permisos

Entidad	Tabla	Función
AccessRequest	access_requests	status(pending/accepted/rejected) — solicitudes de acceso al journal
Connection	connections	Bidireccional (2 filas por par) — accepted/blocked
JournalPermission	journal_permissions	6 flags: canViewStats, canViewTrades, canViewNotes, canViewComments, canDownloadCsv, canViewRealtime
JournalSetting	journal_settings	visibility(public/connections/private)

4.7 Gamificación y Diario

Entidad	Tabla	Función
GameScore	game_scores	mode(classic/survival/daily/arena/torneo/fractal/portfolio/hist), score, xpGained, metadata(json)
DiaryEntry	diary_entries	encryptedData(text AES-256-GCM), iv(48) — diario personal cifrado

Entidad	Tabla	Función
MarketCandle	market_candle	symbol, exchange, interval, open/high/low/close(20,8), volume, timestamp — OHLCV

4.8 Soporte y Sistema

Entidad	Tabla	Función
Notification	notifications	type, content, link, isRead — in-app + push
MonitorEvent	monitor_event	level(error/warning/info), message, stack, source, userCode, url — logging frontend
EconomicReminder	economic_reminders	Recordatorios de eventos económicos
Block	blocks	Usuarios bloqueados

38 Controladores (organizados por dominio)

Controlador	Base URL	Endpoints	Función
DuelController	/api/duels	7	Duelos 1v1 Game APK
SocialController	/api	13	Permisos y conexiones
JournalController	/api/journal	5	CRUD journal con permisos
TournamentController	/api/tournaments	9	Torneos con prize pool
MercadoPagoController	/api/mercadopago	2	Pagos MP
BinancePayController	/api/binance-pay	3	Pagos Binance
CopierController	/api/copier	7	Bridge Python signal copier
AdminCopierController	/api/admin/copier	4	Admin dashboard del copier
AdminWalletController	/api/admin/wallet	6	Credit/debit/withdraw
FeedController	/api/feed	5	Social feed con likes/comments
ChatController	/api/chat	6	Conversaciones DM/grupo/AI
ChatUploadController	/api/chat/upload	1	Upload archivos (20MB max)
UserSoundController	/api/user/sound	2	Preferencia de sonido
AcademiaController	/api/academia	5	CRUD cursos academia
DiaryController	/api/diary	5	CRUD entradas cifradas AES-256-GCM
ProfileController	/api/profile	4	Avatar CRUD + perfil público
GameController	/api/game	5	Game score + leaderboard + XP
GameAppController	/api/app	3	Version info + APK downloads
WalletController	/api/wallet	4	Balance, transactions, withdraw
MarketController	/api/market	6	Prices, exchanges, symbols, candles
MercureController	/api/mercure	2	SSE subscribe URL + auth cookie
MonitoringController	/api/monitoring	3	Log frontend events + stats
MusicController	/api/music	4	Playlist admin + URLs externas
NotificationController	/api/notifications	5	List, markRead, markAllRead, count, delete
LeaderboardController	/api/leaderboard	2	Top 50 traders por PnL
DolarController	/api/wallet/rates	2	ARS exchange rates (cache 1h)
CalendarController	(calendar)	1	Endpoint calendario

Controlador	Base URL	Endpoints	Función
TradingAccountController	/api/accounts	5	Accounts CRUD
JournalController	/api/journal	5	Journal CRUD
DeviceController	/api/devices	2	Register/unregister FCM tokens
FirebaseConfigController	/api/firebase	1	Firebase Web SDK config
AuthController	/api/auth	3	Login/logout/check
AppVersionController	/api/app	2	Version check
EconomicReminderController	/api/economic-reminders	2	Recordatorios eventos
TaskController	/api/tasks	2	List tareas operativas
MainController	/	2	Home + module (renderiza base.html.twig)
DownloadController	/download/tnsvt-market	1	Landing page APK downloads

Total: 115+ endpoints REST JSON en 38 controladores, organizados por dominio funcional.

9 Servicios de Negocio

Servicio	Líneas	Función	Notas
MercadoPagoService	162	Integración MP Checkout Pro (ARS)	file_get_contents + stream_context, Bearer token, IPN URL
BinancePayService	155	Integración Binance Pay (USDT)	HMAC-SHA256 + SHA512 signature verification
PushNotificationService	234	FCM dual-mode (v1 + Legacy)	Service account JWT + OAuth2, fallback a server key
PushService	~200	Kreait SDK + Notification entity	titleForType() mapea 14 tipos
TournamentMailer	~150	Email + push transaccional	Twig emails + broadcast FCM
RealtimePublisher	~80	Publica a Mercure Hub	publish(topic, data, private, type)
MercureSubscriberService	~100	JWT y cookies	getSubscribeUrl, createSubscribeJwt, createAuthCookie
CopierBridgeService	180	Bridge PHP↔Python	receiveTrade, updateTrade, getCopierStatus, getCopierConfig
RateLimiterService	~80	Rate limiter DB-backed	Usado por FeedController, DuelController

15 Migraciones Aplicadas

Fecha	Migración	Contenido
2026-06-10	Version20260610213518	Usuarios iniciales + CodeAuthenticator
2026-06-11	Version20260611003208	Wallet + WalletTransaction
2026-06-12	Version20260612144923	Torneos + TournamentEntry
2026-06-13	Version20260613051000	Feed + FeedPost + LikedPost
2026-06-17	Version20260617171951	Chat + Conversation + Message
2026-06-17	Version20260617192614	Notification entity
2026-06-17	Version20260617193440	Academia + AcademiaContent
2026-06-17	Version20260617235932	Game + GameScore
2026-06-22	Version20260622002308	MercadoPago + Binance Pay services
2026-06-22	Version20260622025850	Push notifications FCM
2026-06-22	Version20260622025909	Music player
2026-06-25	Version20260625021044	Duelos 1v1 (Duel + DuelRound)
2026-06-27	Version20260627140251	Diary AES-256-GCM + traits
2026-06-27	Version20260627214623	Journal social system
2026-07-01	Version20260701115530	Pin/Hash cripto app lock
2026-07-06	Version20260706221000	Trade + TradingAccount unificación

§5 Frontend Web PWA

La SPA usa Twig como motor de plantillas + Vanilla JS modular (sin React/Vue) + Symfony Asset Mapper + Service Worker PWA v57. Todo el JS se carga como ES module via importmap.

base.html.twig — Template Único SPA

El template contiene TODA la UI: login screen, topbar, hub view (visualización SVG con nodos), trading panel con sidebar y 13 tab contents, modals de notificaciones, chat widget CF, music player bar, modales de creación de grupo/DM, y el manifest PWA. Tamaño total: ~4,830 líneas.

Estructura principal

```
<body> ■■■ <div id="login-screen"> // Login con gateKey + admin password ■■■ <div id="main-content"> // Container principal (display:none hasta login) ■ ■■■ <div class="app-header"> // Logo TNSVT + user badge + avatar ■ ■■■ <div id="pushPermBar"> // Banner activar push ■ ■■■ <div id="hub-view"> // Vista hub con SVG nodes (Dios/Psicología/etc) ■ ■■■ <div id="module-panel"> // Panel de módulos del hub ■ ■■■ <div id="trading-panel"> // PANEL PRINCIPAL (display:none hasta triggerCircle) ■ ■ ■■■ <div class="trading-layout"> ■ ■ ■■■ <div class="trading-sidebar"> // 14 botones (sidebar) ■ ■ ■■■ <div class="trading-main"> ■ ■ ■■■ #tab-posts (active default) ■ ■ ■■■ #tab-chart ■ ■ ■■■ #tab-macro ■ ■ ■■■ #tab-2steps-adv ■ ■ ■■■ #tab-tasks ■ ■ ■■■ #tab-calendar ■ ■ ■■■ #tab-journal ■ ■ ■■■ #tab-diary ■ ■ ■■■ #tab-leaderboard ■ ■ ■■■ #tab-academia ■ ■ ■■■ #tab-security ■ ■ ■■■ #tab-social ■ ■ ■■■ #tab-admin ■ ■■■ <!-- CF Chat Widget flotante --> ■ ■■■ <!-- Music player bar (bottom-fixed) --> ■ ■■■ <!-- Modales: NewDM, CreateGroup, ManageGroup --> ■■■ <div id="appLoadingOverlay"> // Loader inicial 12s hard timeout
```

assets/app.js — Módulos principales (6,271 líneas)

Módulo	Líneas aprox	Función
CONFIG & Auth	200	toggleAdminPassField, verifyGateKey, logout, login flow
Hub View	300	Divine Canvas SVG nodes, clickTriggerCircle, learnNodes progression
Trading Panel	500	switchTab, 14 sidebar buttons, 13 tabs
Macroeconomía	900	12 sub-paneles: Bancos, Datos, Ciclo, Gigantes, DotPlot, Geo, Divergencia, Carry, Curva, CicloEco, Herramientas, Quiz
Tareas	100	loadTasks, toggleTask, updateInnerLocks
Calendario	300	Calendario económico + reminders con localStorage
Trading Journal	600	CRUD completo: tjAddTrade, tjEditTrade, tjSetPeriod, tjExportCSV/HTML, tjImport
Feed	400	filterFeed, createNewPost, likeFeedPost, submitComment, initFeedRealtime
Academia	300	renderAcademia, cursos con video y quiz
Music Player	400	Barra persistente estilo Spotify, viz canvas, queue
Push Notifications	200	requestPushPermission, initFCM, getFCMToken, onMessage
Biometric Auth	200	isAvailable, authenticate, isEnabled (PIN/FP)
Diario AES-256-GCM	350	setupPassword, _deriveKey, _encrypt, _decrypt, saveEntry, openReader
Social Module	400	searchUsers, sendAccessReq, loadConnections, updatePerm, loadJournalSettings
CF Chat Widget	1200	DM/grupos, mensajes, adjuntos, sonidos (10 web audio), typing, edit/delete

assets/api.js — API Client (326 líneas)

Cliente fetch wrapper expuesto como window.API. Métodos: request(), get(), post(), put(), del(), patch(). 60+ métodos organizados por dominio con resolución automática de base URL para Capacitor vs browser. Soporta loadingCount para integración con loader de inicialización.

Service Worker PWA v57

- **API calls** — network-first, fallback a cache, respuesta JSON 503 si offline
- **Cache-bust (?v=...)** — network-only (assets versionados siempre frescos)
- **Static assets** — cache-first, fallback a network, runtime cache
- **Push notifications** — event.push con JSON payload, native notification con vibrate [200,100,200]
- **Precache** — index, manifest.json, icons (icon-192, icon-512, maskable-512, apple-touch-icon)

§6 Aplicaciones Android (Web APK + Game APK)

El proyecto incluye DOS aplicaciones Android independientes que comparten el mismo backend Symfony. Ambas usan Capacitor (WebView wrapper) con HTML5/JavaScript embebido. La sección 8 está dedicada enteramente a T.N.S.V.T Market Instinct (Game APK).

6.1 Web APK — com.tnsvt.app

Aspecto	Detalle
Framework	Capacitor v8 (Android)
Package ID	com.tnsvt.app
Versión	v1.6.x (actual 1.6.3, versionCode 9)
Tamaño	268 MB
Build	cd android && gradlew.bat assembleDebug (JAVA_HOME=C:\dev\jdk\jdk-21)
Sync	npx cap sync android
Contenido	Frontend web completo empaquetado (app.js, api.js, chart.js, base.html.twig)
APK	public/downloads/tnsvt-app.apk + public/apk/tnsvt-v1.6.3.apk
Firebase	google-services.json con messaging 23.1.2
Distribución	Tailscale Funnel HTTPS público

6.2 Game APK — com.tnsvt.market.instinct

T.N.S.V.T Market Instinct es la app de juego de trading. Ver **Sección 8 — T.N.S.V.T Market Instinct: el juego** para documentación detallada de modos, duelos, XP y stack técnico.

Aspecto	Detalle
Framework	Capacitor v6 (Android)
Package ID	com.tnsvt.market.instinct
Versión	v1.2.x
Tamaño	5.22 MB
Build	cd game-app/android && gradlew.bat assembleDebug
Contenido	HTML5 canvas game, torneo trading panel, duelos 1v1
APK	public/downloads/tnsvt-market-instinct.apk
Firebase	google-services.json con messaging 23.1.2
Backend	MISMO Symfony compartido (Tournament, Duel, GameScore entities)

6.3 Backend Compartido

AMBAS aplicaciones consumen el MISMO backend Symfony. NO hay backend separado por APK. El Game APK comparte entidades, sistema de pagos, autenticación, push notifications y torneos con el Web APK.

- **Misma entidad Wallet** y WalletTransaction — balance compartido
- **Mismo sistema de pagos** (MercadoPago + Binance Pay)
- **Misma autenticación** (X-Game-Code header)
- **Mismas push notifications** (FCM)
- **Mismo sistema de torneos** (Tournament/TournamentEntry)
- **Mismos duelos 1v1** (Duel/DuelRound)
- **Mismo leaderboard** (LeaderboardController top 50)

§7 T.N.S.V.T Market Instinct: el Juego

T.N.S.V.T Market Instinct es la app de juego de trading incluida en el repositorio tnsvt-symfony como subdirectorio game-app. Es una aplicación Android nativa (APK 5.22 MB) que combina 8 modos de juego, torneos competitivos, duelos 1v1, sistema de XP y rankings, todo respaldado por las entidades Symfony compartidas con el Web APK.

7.1 Identidad y Marca

Aspecto	Detalle
Nombre comercial	T.N.S.V.T Market Instinct
Tagline	Domina tu mente. Regula tu cuerpo. Ejecuta con fe.
Package	com.tnsvt.market.instinct
Versión	v1.2.x
Framework	Capacitor v6 (WebView + HTML5 canvas)
Tamaño APK	5.22 MB
Distribución	public/downloads/tnsvt-market-instinct.apk
Backend	Symfony compartido (mismo que Web APK)
Firebase	google-services.json + messaging 23.1.2

7.4 Duelos 1v1 — Sistema Completo

El modo Arena 1v1 permite a dos usuarios competir en una serie de hasta 5 rondas. Cada jugador toma decisiones de long/short y el sistema compara el PnL real del activo subyacente durante la ronda.

```
Duel (duels) id int (PK) code string(20)
unique # DUEL-A1B2 (auto-generated) entryFee decimal(12,2) prizePool decimal(12,2) # 2 × entryFee (winner
takes all, ties split) totalRounds int default 5 currentRound int default 0 startingPrice decimal(20,8) #
Precio del activo al iniciar player1Pnl decimal(12,2) # PnL acumulado jugador 1 player2Pnl decimal(12,2) # PnL
acumulado jugador 2 status enum # waiting/active/finished/cancelled createdAt DateTimeImmutable
ManyToMany → User (player1) ManyToMany →
User (player2) ManyToMany → User (winner, nullable) OneToMany → DuelRound DuelRound (duel_rounds)
id int (PK) duel_id int (FK) roundNumber
int # 1..totalRounds player1Move enum # long/short/null (no jugó aún) player2Move enum openPrice decimal(20,8)
# Precio al abrir la ronda closePrice decimal(20,8) # Precio al cerrar la ronda highPrice decimal(20,8) # High
durante la ronda lowPrice decimal(20,8) # Low durante la ronda player1Pnl decimal(12,2) # PnL de la ronda para
p1 player2Pnl decimal(12,2) # PnL de la ronda para p2
ManyToMany → Duel Método isBothPlayed() →
true si ambos movieron Método computePnl() → calcula PnL basado en move y OHLC
```

7.5 Flujo de un Duelo 1v1

- **1. Create** — Jugador A crea duelo (POST /api/duels/create) con entryFee y símbolo
- **2. Join** — Jugador B se une (POST /api/duels/{id}/join), se cobra entryFee a ambos
- **3. Play round** — Cada jugador elige long/short (POST /api/duels/{id}/play)
- **4. Next round** — Al completar ambos, se cierra la ronda y se calcula PnL (POST /api/duels/{id}/next-round)
- **5. Winner** — Al completar totalRounds, quien tenga mayor PnL acumulado gana el prizePool
- **Cancel** — Si un jugador no juega en X minutos, el duelo se cancela y se hace refund

7.6 Stack Técnico del Game APK

Capa	Tecnología
UI Framework	HTML5 Canvas 2D + Vanilla JS
Game Engine	Custom game loop con requestAnimationFrame
Build	Capacitor v6 (Android)
Java/Kotlin	JDK 21 + Gradle
Capacitor Plugins	App, Haptics, Keyboard, StatusBar, Push Notifications
Networking	fetch API → TNSVT endpoints (X-Game-Code)
Storage	localStorage para UI prefs (tema, layout)
Auth	Same CodeAuthenticator as Web APK
Telemetry	GameScore entity persistida en TNSVT

7.7 Construcción del APK

Para construir el Game APK desde cero:

```
# Prerrequisitos JDK 21 (C:\dev\jdk\jdk-21\jdk-21.0.7+6) Node.js + npm Android SDK + platform-tools # Build cd
game-app npm install npx cap sync android cd android gradlew.bat assembleDebug # Output #
app/build/outputs/apk/debug/app-debug.apk # Instalar en dispositivo adb install -r app-debug.apk # Publicar
copy app-debug.apk ..\public\downloads\tnsvt-market-instinct.apk
```

7.8 Leaderboard y Competencia

El GameController expone leaderboards globales y por modo. Los rankings se calculan sobre XP acumulado y score más alto por modo. Las rachas (streaks) y badges se desbloquean automáticamente.

- **GET /api/leaderboard** — top 50 traders globales por PnL
- **GET /api/game/my-stats** — XP, rank, mejores scores por modo del usuario actual
- **GET /api/leaderboard?mode=arena** — top por arena 1v1 (wins/losses)
- **GET /api/leaderboard?mode=torneo** — top por torneos (championships)
- **POST /api/game/save-score** — registra score de partida completada

§8 Sistema Signal Copier Python

El Signal Copier Python es el cerebro automatizado del sistema. Escucha canales de Telegram, parsea señales, las ejecuta en MetaTrader 5 con gestión de riesgo, y sincroniza el journal con TNSVT en tiempo real.

8.1 Stack y Arquitectura

Aspecto	Detalle
Python	3.12
Telegram Listener	Telethon (asyncio, MTProto)
MT5 Bridge	MetaTrader5 library oficial
Local DB	SQLite (trades.db)
HTTP Client	requests + tnsvt_client.py
Logging	Python logging + signal_copier.log
Async	asyncio (news_closer, config_watcher, tnsvt_heartbeat, mt5_trade_monitor)
Total archivos	13 .py en signal_copier/
Líneas main.py	341

8.2 Estructura de Archivos

```
Terminal_Financiera_Pro/ ■■■ bot/ # Telegram bot (11 comandos) ■ ■■■ main.py ■ ■■■ handlers/ # start, calendar,
news, signals, admin ■ ■■■ services/ # markets, jblanked, news_api, trading_economics, tnsvt_status ■■■
signal_copier/ # EL COPIADOR PRINCIPAL ■ ■■■ main.py # 341 líneas - entry point ■ ■■■ parser.py # SignalParser
multi-formato ■ ■■■ executor.py # MT5Executor + MT5Monitor ■ ■■■ risk_manager.py # Límites diarios/semanales ■ ■■■
news_filter.py # Filtro noticias NFP/FOMC ■ ■■■ database.py # SQLite local trades.db ■ ■■■ session # Telethon auth
session ■■■ tradingeconomics/ # SDK wrapper ■■■ api_server.py # FastAPI bridge (port 8502) ■■■ dashboard/app.py #
Streamlit dashboard ■■■ tnsvt_client.py # Cliente HTTP para TNSVT ■■■ config/settings.py # Settings centralizado
■■■ login_telegram.py # Auth helper Telethon
```

8.3 SignalParser — Multi-formato

El parser reconoce señales en CUALQUIER formato común de Telegram:

```
BUY_WORDS = [ "comprar", "compra", "buy", "long", "abrir compra", "abrir long", "entrada compra", "entrada long", "buy now", "buy now!", "comprar ahora", "ir long", "ir a long", "buy:" ]
SELL_WORDS = [ "vender", "venta", "sell", "short", "abrir venta", "abrir short", "entrada venta", "entrada short", "sell now", "sell now!", "vender ahora", "ir short", "ir a short", "sell:" ]
CLOSE_WORDS = [ "cerrar", "cierre", "close", "salir", "salida", "cerrar todo", "close all", "exit", "cambiar", "revertir" ]
def parse_message(text: str, pending_signal: dict = None) -> dict:
    signal = { "action": None, "symbol": None, "price": None, "price_range": None, # Para rangos como "2990.50 - 2992.50"
               "sl": None, "tp": [], "lot": None, "is_update": False, "tp_percentages": [], }
    # 1. Verificar si es solo SL/TP update de señal pendiente
    # 2. Detectar acción (BUY/SELL/CLOSE)
    # 3. Extraer símbolo (EURUSD, BTCUSDT, XAUUSD, etc)
    # 4. Extraer SL/TP (múltiples formatos: 50 pips, 1.0500, etc)
    # 5. Detectar lot size si está presente
    return signal
```

8.4 MT5Executor

Ejecuta órdenes en MetaTrader 5 con magic number 20260706 para identificar señales del copiador:

```
class MT5Executor:
    def __init__(self):
        self.connected = False
        self.last_order_ticket = None # Para trade_monitor map
    async def execute_trade(self, signal: dict) -> bool:
        # 1. connect() si no está conectado
        # 2. _ensure_symbol(symbol) - busca variaciones (.Raw, .r, etc)
        # 3. _get_default_lot(symbol) - calcula lot según LOT_MODE
        # 4. _get_price(symbol) - bid/ask/spread
        # 5. order_send con magic=20260706
        # 6. partial closes si hay múltiples TP
        return True
    async def _close_positions(self, symbol: str = None) -> bool:
        # Cierra posiciones con cierre inverso
```

8.5 RiskManager

```
class RiskManager:
    def __init__(self):
        self.config = {
            "daily_loss_limit": settings.RISK_DAILY_LOSS_LIMIT, # 2.0%
            "daily_profit_target": settings.RISK_DAILY_PROFIT_TARGET, # 5.0%
            "weekly_loss_limit": settings.RISK_WEEKLY_LOSS_LIMIT, # 5.0%
            "max_open_positions": settings.RISK_MAX_OPEN_POSITIONS, # 5
            "trailing_stop": settings.RISK_TRAILING_STOP, # true
            "trailing_step": settings.RISK_TRAILING_STEP, # 10 pips
            "trailing_start": settings.RISK_TRAILING_START, # 50 pips
        }
    def can_open_trade(self) -> tuple[bool, str]:
        # Verifica daily_loss_limit, weekly_loss_limit, max_open_positions
        # Retorna (True, None) si puede, (False, reason) si no
        return True, None
    def check_trailing_stop(self, symbol: str) -> None:
        # Ajusta SL de posiciones abiertas según trailing
```

8.6 NewsFilter

Filtra trades alrededor de noticias económicas de alto impacto:

```
HIGH_IMPACT_KEYWORDS = [ "NFP", "NON-FARM", "PAYROLL", "CPI", "PPI", "GDP", "FOMC", "FED", "INTEREST RATE",
"RATE DECISION", "RETAIL SALES", "PMI", "ISM", "UNEMPLOYMENT", "INFLATION", "CONSUMER PRICE", "PRODUCER
PRICE", "TRADE BALANCE", "HOUSING STARTS", "BUILDING PERMITS", "DURABLE GOODS", "INITIAL JOBLESS", "CONTINUING
CLAIMS", "FED CHAIR", "POWELL", "YELLEN", "LAGARDE", "ECB", "BOJ", "BOE", "SNB", "RBA", "RBNZ", ]
class NewsFilter:
def should_block_trade(self) -> tuple[bool, str]: # Retorna (True, reason) si hay noticia dentro
de NEWS_FILTER_MINUTES return False, None
def get_positions_to_close(self) -> list: # Retorna símbolos a
cerrar antes de noticia return []
```

8.7 mt5_trade_monitor — Loop asíncrono (Fase 4)

Cada 10 segundos verifica posiciones abiertas en MT5. Cuando detecta que un ticket ya no está en positions_get(), consulta history_deals_get() para obtener el PnL real y actualiza TNSVT.

```
async def mt5_trade_monitor(): """Monitorea trades y actualiza PnL en TNSVT al cierre""" while True: await
asyncio.sleep(10) if not tnsvt_client.enabled or not executor.connected: continue if not trade_map: continue
positions = mt5.positions_get(magic=20260706) or [] open_tickets = {int(p.ticket) for p in positions} for
ticket_str in list(trade_map.keys()): ticket = int(ticket_str) if ticket not in open_tickets: # Trade cerrado
- consultar PnL real entry = trade_map.pop(ticket_str) trade_id = entry.get("trade_id") deals =
mt5.history_deals_get(since, datetime.now(), position=ticket) or [] pnl = sum(d.profit + d.swap + d.commission
for d in deals if d.position == ticket and d.entry == mt5.DEAL_ENTRY_OUT) result_label = "WIN" if pnl > 0 else
("LOSS" if pnl < 0 else "BREAKEVEN") tnsvt_client.update_trade(trade_id=trade_id, result=result_label,
pnl=pnl)
```

8.8 Tasks Asíncronos en main.py

```
async def main(): init_db() mt5_connected = executor.connect() await client.connect() if not await
client.is_user_authorized(): print("Sesión no autorizada. Ejecuta: python login_telegram.py") return # Tasks
paralelos: asyncio.create_task(config_watcher()) # Vigila config/reload.trigger (2s poll)
asyncio.create_task(news_closer()) # Cierra posiciones antes de noticias (60s poll)
asyncio.create_task(tnsvt_heartbeat()) # POST /api/copier/status (30s)
asyncio.create_task(mt5_trade_monitor()) # Detecta cierres y actualiza TNSVT (10s) await
client.run_until_disconnected()
```

★ **NOTA:** Fase 4 — Auto-update PnL: el trade_map se persiste en var/tnsvt_trade_map.json para sobrevivir reinicios del proceso. El merge de status preserva los campos del bot (telegram_bot, bot_username) enviados por el Telegram Bot.

§9 Telegram Bot

Bot de Telegram con 11 comandos que combina servicios externos (Yahoo Finance, NewsAPI, TradingEconomics, JBlanked) con la API TNSVT. Después de la Fase 3, los comandos /stats y /senales consultan TNSVT en vez de SQLite local. El bot envía heartbeat cada 30s para mantener su presencia en el dashboard.

9.1 Comandos

Comando	Servicio	Función
/start, /help	start.py	Muestra menú completo de comandos
/mercados	markets.py	Resumen S&P500;, Nasdaq, Dow, VIX, FTSE, DAX, CAC, IBEX
/cripto	markets.py	Precios crypto desde JBlanked
/noticias	news_api.py	Noticias de mercados (NewsAPI)
/ipc	news_api.py	Noticias de inflación
/morgan	news_api.py	Noticias de Wall Street
/calendario, /datos	trading_economics.py	Eventos económicos + indicadores macro
/senales	tnsvt_status.py (Fase 3)	Estado del signal copier desde TNSVT
/resumen	news_api.py	Resumen diario (IPC, empleo, Morgan, crypto)
/stats	tnsvt_status.py (Fase 3)	Estadísticas del copier desde TNSVT API

9.2 Servicios Externos

Servicio	API	Uso
Markets	Yahoo Finance API (query1.finance.yahoo.com)	Resumen de índices globales
JBlanked	jblanked.com/news/api/list	Calendario económico + crypto
NewsAPI	newsapi.org/v2/everything	Búsqueda de noticias por query
TradingEconomics	tradingeconomics SDK	Indicadores macro por país (ARG, USA)
TNSVT API	laptop-ebgqig6j.tailf43f87.ts.net	Estado del copier + heartbeat

9.3 Integración TNSVT (Fase 3) — tnsvt_status.py

Módulo wrapper ligero que cachea respuestas de TNSVT API para no saturar el servidor. Usado por /senales y /stats.

```
from tnsvt_client import TNSVTClient def get_copier_status_from_tnsvt() -> Optional[dict]: """Retorna status del signal_copier desde TNSVT.""" client = get_client() # Singleton return _cached("status", lambda: client.get_dashboard()) def get_copier_stats_from_tnsvt() -> dict: """Stats consolidadas: balance, pnl, winrate, mt5""" return _cached("stats", lambda: _fetch_stats(client), ttl=10) def send_bot_heartbeat(bot_username: str) -> bool: """Marca el bot como vivo en status TNSVT.""" return client.update_status_field(telegram_bot=True, bot_username=bot_username)
```

9.4 Heartbeat del Bot

Loop asíncrono que cada 30s actualiza el status TNSVT con el campo telegram_bot=true:

```
async def tnsvt_heartbeat_loop(bot_app): bot_username = "telegram_bot" try: me = await bot_app.bot.get_me() if me and me.username: bot_username = me.username except Exception: pass while True: try: send_bot_heartbeat(bot_username) except Exception as e: logger.error(f"Heartbeat error: {e}") await asyncio.sleep(30)
```

§10 TNSVT Bridge — FastAPI (port 8502)

Servidor FastAPI asíncrono que permite a TNSVT enviar señales al copiador de forma remota. Recibe señales HTTP, las encola en `asyncio.Queue`, y `signal_copier` las consume cada N segundos.

10.1 Endpoints

Endpoint	Método	Auth	Función
/api/signals	POST	ninguna	TNSVT encola señal para copiar
/api/signals/pending	GET	X-Admin-Password	Copier consulta señales pendientes
/api/status	GET	X-Admin-Password	Estado del copier (MT5, news_filter, pnl)
/api/config	GET	X-Admin-Password	Configuración actual del .env
/api/config	PUT	X-Admin-Password	Actualiza config + reload.trigger
/health	GET	ninguna	Health check simple

10.2 Implementación clave

```
import asyncio from fastapi import FastAPI, HTTPException, Header from pydantic import BaseModel
ADMIN_PASSWORD = os.getenv("TNSVT_ADMIN_PASSWORD", "") SIGNAL_QUEUE: asyncio.Queue = asyncio.Queue()
class SignalPayload(BaseModel):
    symbol: str
    action: str
    price: Optional[float] = None
    sl: Optional[float] = None
    tp: Optional[list[float]] = None
    lot: Optional[float] = None
    channel: str = "TNSVT"
def verify_admin(x_admin_password: str = Header(default="")):
    if not ADMIN_PASSWORD:
        raise HTTPException(500, "TNSVT_ADMIN_PASSWORD not configured")
    if x_admin_password != ADMIN_PASSWORD:
        raise HTTPException(403, "Invalid admin password")
@app.post("/api/signals")
async def receive_signal(signal: SignalPayload):
    await SIGNAL_QUEUE.put(signal.model_dump())
    return {"status": "queued", "symbol": signal.symbol}
@app.put("/api/config")
async def update_config(config: ConfigPayload, x_admin_password: str = Header(default="")):
    verify_admin(x_admin_password)
    updates = {k: v for k, v in config.model_dump().items() if v is not None}
    env_map = {"channels": "CHANNELS", "lot_size": "LOT_SIZE", ...}
    env_updates = {env_map.get(k): str(v) for k, v in updates.items() if env_map.get(k)}
    settings.save(env_updates)
    trigger_path = ROOT_DIR / "config" / "reload.trigger"
    trigger_path.write_text(str(asyncio.get_event_loop().time()))
    return {"success": True}
```


§11 Streamlit Dashboard (port 8501)

Dashboard independiente construido con Streamlit y Pandas para visualizar el estado del signal_copier. Usa JetBrains Mono para tipografía monoespaciada y tema dark con acentos cyan/blue.

11.1 Estructura

```
import streamlit as st from signal_copier.database import init_db, get_trades, get_stats, get_trades_today
from signal_copier.risk_manager import RiskManager from signal_copier.news_filter import NewsFilter
st.set_page_config( page_title="Terminal Financiera Pro", page_icon="■", layout="wide",
initial_sidebar_state="collapsed", ) # DARK CSS (Inter + JetBrains Mono) DARK_CSS = """<style>...</style>""" #
4 TABS principales: tab1, tab2, tab3, tab4 = st.tabs(["■ Dashboard", "■ Operaciones", "■■ Riesgo", "■■
Configuracion"])
```

11.2 Métricas Mostradas

Tab	Métricas	Datos
Dashboard	TOTAL, EJECUTADAS, BLOQUEADAS, HOY, WIN RATE, GANADAS, PERDIDAS, PNL TOTAL	get_stats() + get_trades_today()
Operaciones	Tabla de trades (Fecha, Símbolo, Acción, Resultado, Canal, PnL)	get_trades()
Riesgo	PNL DIARIO, PNL SEMANAL, TRADES HOY, BALANCE, LIMITE DIARIO/SEMANAL, MAX POSICIONES	risk_manager.get_status()
Configuración	Settings panel + estado de risk_manager y news_filter	settings + RiskManager + NewsFilter

§12 Integración PHP↔Python (Fases 1-4)

La integración entre el backend Symfony (PHP) y el sistema Python se construyó en 4 fases incrementales, cada una agregando una capa de comunicación bidireccional.

12.1 Fase 1 — API Bridge (Endpoints REST)

Capa base: tnsvt_client.py en Python + CopierBridgeService + CopierController + AdminCopierController en PHP.

```
# tnsvt_client.py (Python side)
class TNSVTClient:
    def log_trade(self, symbol, action, price, sl, tp, result, pnl, channel, account_id) -> int:
        """POST /api/copier/trades → crea Trade entity con notes='Auto-copied from: <channel>'"""
    def update_trade(self, trade_id, result=None, pnl=None, sl=None, tp=None) -> bool:
        """PUT /api/copier/trades/{id} → actualiza trade existente"""
    def send_heartbeat(self, status_data: dict) -> bool:
        """POST /api/copier/status → escribe JSON a var/copier_status.json"""
    def get_dashboard(self) -> Optional[dict]:
        """GET /api/admin/copier/dashboard → status + config + recent_trades"""
    def get_recent_trades(self, limit: int = 50) -> list:
        """GET /api/admin/copier/trades → últimos N trades del journal"""
    def update_status_field(self, **fields) -> bool:
        """Merge fields en status existente (para preservar telegram_bot)"""
```

12.2 Fase 2 — Admin Copier Dashboard

Subtab 'Copier' en el panel admin con status cards en vivo (MT5, PnL Diario, Balance, Trades Hoy, Win Rate, Telegram Bot). Refresca cada vez que el usuario hace click.

[illegible]

12.3 Fase 3 — Bot TNSVT

Telegram Bot consulta TNSVT API en vez de SQLite local. Heartbeat cada 30s mantiene campo telegram_bot=true en status.

- /stats → consulta TNSVT dashboard (balance, pnl, winrate)
- /senales → consulta TNSVT status + recent_trades (lista últimas 5)
- send_bot_heartbeat() → preserva merge con status del signal copier

12.4 Fase 4 — Auto-Update PnL

mt5_trade_monitor detecta cierres de posición en MT5, consulta history_deals_get para PnL real, y actualiza TNSVT journal con result=WIN/LOSS y pnl=X. El mapeo ticket→trade_id persiste en disco.

```
# var/tnsvt_trade_map.json (persiste entre reinicios) { "12345678": { "trade_id": 42, "symbol": "EURUSD",  
"action": "BUY", "channel": "Señales Vip", "opened_at": 1717670000.123 }, "87654321": { "trade_id": 43,  
"symbol": "XAUUSD", "action": "SELL", "channel": "INVESTMENTH VIP", "opened_at": 1717670123.456 } }
```

★ **NOTA:** Merge de status: el signal_copier preserva telegram_bot=true y bot_username en cada heartbeat, evitando que el bot parezca offline después de un trade.

§13 Seguridad

El sistema implementa múltiples capas de seguridad: autenticación por código, headers de admin, firmas HMAC para webhooks de pago, JWT para Mercure, biometrics en APK.

13.1 CodeAuthenticator

```
class CodeAuthenticator extends AbstractAuthenticator: def authenticate(Request $request): Passport $code = $request->headers->get('X-Game-Code') if (!$code) { $code = $request->request->get('user_code'); } $user = $this->userRepository->findByCode($code); if (!$user || !$user->isActive()) { throw new AuthenticationException('Invalid code'); } return new SelfValidatingPassport(new UserBadge($code));
```

13.2 AdminAuthTrait

Trait usado por todos los controllers admin que validan X-Admin-Password:

```
trait AdminAuthTrait { protected function requireAdmin(Request $request): void { $password = $_ENV['ADMIN_PASSWORD'] ?? ''; $headerPass = $request->headers->get('X-Admin-Password'); if ($headerPass !== $password) { throw new AccessDeniedHttpException('Invalid admin password'); } }
```

13.3 Firmas y Cifrado

Mecanismo	Algoritmo	Uso
Binance Pay webhook	HMAC-SHA512	Verifica BINANCE_PAY_SECRET_KEY en /api/binance-pay/webhook
MercadoPago	HMAC-SHA256	Atomic SQL en webhook para idempotencia
Mercure subscribe	JWT RS256	mercure.subscribe claim, 1h expiración
Android biometrics	BiometricPrompt + PIN	Plugin Java BiometricPlugin con Capacitor 8/6
Diario AES-256-GCM	AES-256-GCM	Cifrado client-side, key derivada de password
network_security_config	TLS only	Cleartext deshabilitado por defecto, solo dev IPs

§14 Sistema de Pagos Compartido

★ **NOTA:** MercadoPago y Binance Pay son parte del BACKEND SYMFONY COMPARTIDO. NO son exclusivos del Game APK ni del Web APK. Ambos APKs pueden usar cualquiera de los dos métodos de pago.

14.1 MercadoPago (ARS)

- Cliente solicita POST /api/mercadopago/create-payment con amount_usd (1-1000)
- Backend convierte USD a ARS via DolarController (dolarapi.com, cache 1h)
- MercadoPagoService::createPreference() crea preferencia en MP API
- Retorna init_point URL. Cliente abre en WebView o browser
- MP envía IPN a POST /api/mercadopago/webhook (verificación HMAC)
- processPaymentNotification() valida, acredita wallet con SQL atómico, marca WalletTransaction
- Idempotente: verifica refPaymentId duplicado

14.2 Binance Pay (USDT)

- Cliente POST /api/binance-pay/create-order con amount_usd
- BinancePayService::createOrder() crea orden con HMAC-SHA256
- Retorna checkoutUrl. Cliente abre en WebView
- Binance envía IPN con HMAC-SHA512 signature a /api/binance-pay/webhook
- verifyWebhookSignature() valida contra BINANCE_PAY_SECRET_KEY
- Si PAY_SUCCESS → acredita wallet, actualiza WalletTransaction

14.3 WalletTransaction — Tipos y Estados

```
TYPES: TYPE_DEPOSIT = "deposit" # Crédito manual/admin TYPE_ENTRY_FEE = "entry_fee" # Fee de torneo
TYPE_PAYOUT = "payout" # Premio de torneo TYPE_REFUND = "refund" # Reembolso TYPE_WITHDRAW = "withdraw" #
Retiro solicitado TYPE_DUEL_ENTRY = "duel_entry" # Fee de duelo 1v1 TYPE_DUEL_WIN = "duel_win" # Premio de
duelo TYPE_DUEL_REFUND = "duel_refund" # Reembolso de duelo STATUSES: STATUS_PENDING = "pending" # Pendiente
STATUS_CONFIRMED = "confirmed" # Confirmada STATUS_REJECTED = "rejected" # Rechazada STATUS_REFUNDED =
"refunded" # Reembolsada METHODS: MANUAL_MP, MANUAL_BINANCE, MANUAL_CRYPT0, AUTO_MP, AUTO_BINANCE,
AUTO_CRYPT0, GIFT, OTHER
```

§15 Real-time (Mercure) + Push (FCM)

15.1 Mercure — Server-Sent Events

Sistema de tiempo real basado en Mercure (SSE) para streaming de velas 15m y tickers:

```

MercureStreamCommand
RealtimePublisher
Mercure Hub
(PHP daemon 3s)
Symfony Service
Docker :3000
SSE Stream
/chart/{ex}/{s}
Web APK PWA
Game APK
chart.js
HTML5 Canvas
EventSource
EventSource
```

15.2 MercureStreamCommand — Daemon

```
bin/console mercure:stream-candles # Loop: cada 3 segundos: # 1. Fetch últimas 3 velas (interval 15m) desde
Binance API # 2. Para 18 symbols en 3 exchanges (binance, bybit, kraken) # 3. Publica por symbol a
/chart/{exchange}/{symbol} # 4. Publica snapshot agregado a /chart/ticker
```

15.3 Push Notifications FCM

Componente	Función
PushNotificationService	234 líneas, dual-mode (v1 + Legacy), OAuth2 service account JWT
PushService (alternativo)	Kreait Firebase Admin SDK + Notification entity persistida
FCM v1 API	POST https://fcm.googleapis.com/v1/projects/{id}/messages:send
FCM Legacy	POST https://fcm.googleapis.com/fcm/send con server key (fallback)
Tipos notificación	comment, like, post, mention, signal, dm, academia, task, access_request, access_accepted, access_rejected, connection_removed, permissions_changed

§16 Journal Social System

Sistema completo de permisos y conexiones para compartir el journal de trading de forma selectiva. Permite buscar usuarios, solicitar acceso, aceptar/rechazar, configurar permisos granulares y controlar la visibilidad del propio journal.

16.1 Entidades (4)

Entidad	Tabla	Función
AccessRequest	access_requests	status(pending/accepted/rejected/cancelled) — solicitudes de acceso
Connection	connections	Bidireccional (2 filas por par) — accepted/blocked
JournalPermission	journal_permissions	6 flags booleanos granulares por par user
JournalSetting	journal_settings	visibility(public/connections/private)

16.2 Permisos Granulares (6 flags)

```
class JournalPermission: can_view_stats bool # Ver estadísticas básicas (winrate, PnL total) can_view_trades bool # Ver trades individuales can_view_notes bool # Ver notas/análisis del trade can_view_comments bool # Ver comentarios en trades can_download_csv bool # Descargar journal como CSV can_view_realtime bool # Ver journal en tiempo real
```

16.3 Niveles de Visibilidad

- **public** — Todos ven estadísticas básicas (nombre, winrate, PnL aggregate)
- **connections** — Solo usuarios aceptados ven trades (filtro por permisos)
- **private** — Solo el owner puede ver su journal

16.4 Endpoints principales

- POST /api/access-request — enviar solicitud de acceso
- GET /api/access-request — listar pendientes (received/sent)
- PATCH /api/access-request/{id} — accept/reject (crea connections bidireccionales)
- GET /api/connections — listar conexiones
- POST /api/connections/{id}/block — bloquear
- GET /api/permissions/{code} — ver permisos de un usuario
- PATCH /api/permissions/{code} — actualizar permisos
- GET /api/journal/settings, PATCH /api/journal/settings — visibilidad
- GET /api/access-status/{code} — estado de la relación (none/pending/connected)

§17 Gamificación

Sistema integral de motivación que combina academia (cursos con progreso), XP en juego, macroeconomía interactiva con quiz, y el Divine Canvas visual del hub.

17.1 Academia (6 cursos)

Curso	Módulo	Contenido
1. Psicología e Identidad Anclada	psi	Fractura de identidad, colapso biológico, observador no reactivo, regulación vagocelular
2. Análisis Técnico Algorítmico	tec	Huella institucional, mapeo de estructuras, liquidez, killzones
3. Análisis Fundamental & Flujo Macro	fun	PMI, indicadores adelantados, diferencial de tasas
4. Niveles OTE Sagrados	fib	Zonas óptimas de retroceso, premium vs discount, confluencia
5. Lógica de Ejecución (2 Steps)	step	Simplificación, modelo madre, BOS, LG
6. Quiz completo	quiz	10 preguntas evaluación final

17.2 Macroeconomía (12 sub-paneles)

- ■ Bancos Centrales (FED, BCE, BOJ) · ■ Soft/Hard Data · ■ Ciclo Mensual
- ■ Gigantes (NFP, CPI, PCE, PMI, FOMC) · ■ Dot Plot · ■ Geopolítica
- ■ Divergencia PMI · ■ Carry Trade · ■ Curva de Rendimientos
- ■ Ciclo Económico · ■ Herramientas · ■ Quiz 10 preguntas

17.3 Divine Canvas Hub

Vista hub central con SVG nodes que representan los 6 pilares del conocimiento. Click en cada nodo abre el módulo correspondiente. Click en el hexágono central activa clickTriggerCircle() que abre el trading panel.

§18 Deployment + Acceso Remoto

El sistema se ejecuta localmente con PHP en background y servicios Python. Tailscale Funnel expone la URL pública HTTPS para acceso desde cualquier dispositivo.

18.1 Scripts de Arranque (.bat)

Script	Comando	Levanta
start_all.bat	Arranca los 4 servicios en ventanas separadas	Bot + Copier + Dashboard + Bridge
start_bot.bat	Solo Telegram bot	python -m bot.main
start_copier.bat	Solo signal copier	python -m signal_copier.main
start_dashboard.bat	Solo Streamlit	streamlit run dashboard/app.py --server.port 8501
start_bridge.bat	Solo FastAPI bridge	python api_server.py

18.2 Tailscale Funnel

```
# Configuración inicial tailscale set --accept-routes tailscale funnel --bg --yes 8000 # Verificar estado  
tailscale funnel status # https://laptop-ebgqig6j.tailf43f87.ts.net (Funnel on) # |-- / proxy  
http://127.0.0.1:8000 # PHP server debe escuchar en 0.0.0.0:8000 php -S 0.0.0.0:8000 -t public
```

18.3 Variables de entorno críticas (.env)

Variable	Valor	Uso
TNSVT_URL	https://laptop-ebgqig6j.tailf43f87.ts.net	Base URL para tnsvt_client.py
TNSVT_USER_CODE	DEMO	Default user_code para signal_copier
TNSVT_ADMIN_PASSWORD	TNSVT-2026-CristoRey!	Auth admin endpoints
BRIDGE_PORT	8502	FastAPI bridge port
MP_ACCESS_TOKEN	(secreto)	MercadoPago API
BINANCE_PAY_API_KEY	(secreto)	Binance Pay
BINANCE_PAY_SECRET_KEY	(secreto)	Webhook signature
MERCURE_JWT_SECRET	(secreto)	Mercure publisher/subscriber
FCM_SERVICE_ACCOUNT	(path JSON)	Firebase v1 API
BOT_TOKEN	(secreto)	Telegram bot

Variable	Valor	Uso
TELETHON_API_ID/HASH	(secreto)	Telethon listener
ADMIN_PASSWORD	TNSVT-2026-CristoRey!	Admin auth trait

18.4 Firebase Project

- Project ID: juego-app-store-trading
- Service Account: android/app/juego-app-store-trading-firebase-adminsdk-fbsvc-81993dce61.json
- google-services.json package: T.N.S.V.TMarketInstic (Game) + com.tnsvt.app (Web)

§19 Roadmap y Métricas Finales

19.1 Commits Recientes

Repo	Commit	Descripción
tnsvt-symfony	f0ba74d	fix(ui): tabs invisibles (closing divs faltantes en tab-macro)
tnsvt-symfony	3555a77	feat: Telegram Bot status card en admin Copier dashboard
tnsvt-symfony	724877b	fix: remove duplicate adminSubtabContentCopier div
tnsvt-symfony	1e3564c	feat: signal copier bridge Phase 2 dashboard integration
TNSVT-SISTEM A-COPY	119a09c	fix(copier): preserve telegram_bot field in heartbeat
TNSVT-SISTEM A-COPY	269cc0e	feat(Fase 3): bot integrado con TNSVT API
TNSVT-SISTEM A-COPY	dd1e1ec	feat(Fase 4): auto-update PnL en TNSVT al cerrar posición MT5
TNSVT-SISTEM A-COPY	8681404	feat: enhanced heartbeat with channels, MT5 status, news filter

19.2 Líneas de Código por Módulo

Módulo	Líneas	Archivo
Backend PHP (Symfony)	~12,000	src/ + templates/
Frontend JS	6,271	assets/app.js
API Client JS	326	assets/api.js
CSS	~3,000	assets/styles/app.css
Twig template	~4,830	templates/base.html.twig
Signal Copier main	341	signal_copier/main.py
Signal Copier total	~2,500	13 archivos .py
Telegram Bot	~1,800	bot/ + services/
FastAPI Bridge	152	api_server.py
Streamlit Dashboard	304	dashboard/app.py
TNSVT Client	221	tnsvt_client.py
PDF Generator	~1,500	generate_full_system_pdf.py
TOTAL	~32,500	Sistema completo

19.3 Estado del Proyecto

Funcionalidad	Estado	Notas
Backend Symfony	■ Completo	29 entidades, 38 controllers, 9 services
Frontend PWA	■ Completo	6,271 líneas JS, 14 sidebar buttons, 13 tabs
Web APK	■ v1.6.3	Capacitor v8, 268 MB
Game APK (Market Instinct)	■ v1.2.x	Capacitor v6, 5.22 MB, 8 modos
Telegram Bot	■ Completo	11 comandos, integración TNSVT
Signal Copier Python	■ Completo	Telethon + MT5, risk manager, news filter
TNSVT Bridge FastAPI	■ Completo	Cola async, HMAC auth
Streamlit Dashboard	■ Completo	4 tabs, métricas live
MercadoPago	■ Completo	ARS + HMAC webhook
Binance Pay	■ Completo	USDT + HMAC-SHA512
FCM Push	■ Completo	v1 API + Legacy, 14 tipos
Mercure Real-time	■ Completo	SSE velas 15m
Journal Social	■ Completo	6 permisos + 3 visibilidades
Tailscale Funnel	■ Activo	URL pública HTTPS
PostgreSQL migration	■ Pendiente	Actual: SQLite dev
SMTP real	■ Pendiente	Actual: null://null dev
iOS version	■ Largo plazo	Capacitor iOS
Multi-idioma (EN/PT)	■ Largo plazo	

19.4 Repositorios GitHub

- **Backend Symfony:** <https://github.com/federicocasal73-gif/tnsvt-symfony>
- **Signal Copier Python:** <https://github.com/federicocasal73-gif/TNSVT-SISTEMA-COPY>
- **URL pública:** <https://laptop-ebgqig6j.tailf43f87.ts.net/>
- **Web APK:** public/downloads/tnsvt-app.apk + public/apk/tnsvt-v1.6.3.apk
- **Game APK:** public/downloads/tnsvt-market-instinct.apk



T.N.S.V.T

Sistema Copy

Fin del documento

Versión 1.0 · Generado: 06/07/2026

Backend: github.com/federicocasal73-gif/tnsvt-symfony

Signal Copier: github.com/federicocasal73-gif/TNSVT-SISTEMA-COPY

URL pública: <https://laptop-ebgqig6j.tailf43f87.ts.net/>

Domina tu mente. Regula tu cuerpo. Ejecuta con fe.